

Siber Güvenlik Mühendisliği Eğitimi – 2026

101 Bitirme Projesi

"The Analyst & The Hunter"

Hazırlayan:

Şeymanur Yılmaz

Tarih:

11 Mart 2026

2. FAZ 1 –Siber Triatlon Operasyon Raporu

2.1 Bölüm A –Teori ve Mimari

2.1.1 SSH Anahtarları

2.1.2 Hex Dump

2.1.3 Client Side vs Server Side

2.1.4 Command Injection

2.1.5 C2 Sunucuları

2.1.6 PCAP Analizi

2.2 Bölüm B –Saha Operasyonu

2.2.1 Bandit 11-15

2.2.2 Natas 0-10

2.2.3 C2 Carnage

3. FAZ 2 –Gerçek Saha Operasyonu (Recon)

BÖLÜM A

1. İşletim Sistemi ve Şifreleme: "Anahtarlar Kimde?"

SSH Private Key (Gizli Anahtar): SSH anahtarları, sunuculara güvenli bir şekilde bağlanmak için kullanılan bir kimlik doğrulama yöntemidir. Parola ile giriş yapılabilse de güvenlik açısından çoğu sistemde SSH anahtarları tercih edilir. Bu sistemde public key (açık anahtar) ve private key (gizli anahtar) olmak üzere iki anahtar bulunur. Parola ile giriş yapıldığında girilen şifre sunucudaki hash değeri ile karşılaştırılır. Ancak bu yöntem brute-force (kaba kuvvet) saldırılarına karşı daha savunmasız olabilir. SSH anahtar yapısında ise asimetrik şifreleme kullanılır. Bu yöntemde public key sunucuya eklenir, private key ise kullanıcıda saklanır. Bağlantı sırasında sunucu, private key'e sahip olan kullanıcının gerçekten yetkili olduğunu doğrular ve böylece parola göndermeye gerek kalmadan güvenli bir bağlantı sağlanır.

Hex Dump: Bir .exe veya binary dosyasını Not defteriyle açmaya çalışırsak anlamsız karakterler görürüz. Hex Dump, bu veriyi standart 16'lık (hexadecimal) sisteme çevirerek dosya başlıklarını (Magic Bytes) görmemizi sağlar. Hex dump sayesinde dosya içeriğindeki byte değerleri incelenebilir ve gerektiğinde bu veriler tekrar normal formata dönüştürülebilir.

2. Web Mimarisi: "Sunucu Tarafı (Server-Side)"

Client-Side vs Server-Side: Client-side kodlar kullanıcının tarayıcısında çalışan kodlardır. HTML, CSS ve JavaScript bu kategoriye girer. Bu kodlar tarayıcı tarafından indirildiği için kullanıcı "Kaynağı incele" gibi araçlarla bu kodları görebilir. Tarayıcı bir web sayfasını oluşturabilmek için bu kodları okumak zorundadır; aksi halde sayfadaki butonları, metinleri veya tasarımı gösteremez. Server-side kodlar ise sunucu tarafında çalışan kodlardır ve kullanıcıya doğrudan gösterilmez. PHP, Python ve NodeJS gibi diller bu kategoriye girer. Kullanıcı bir istekte bulunduğunda sunucu bu isteği alır, ilgili server-side kodu çalıştırır ve işlem sonucunda oluşan HTML çıktısını kullanıcıya gönderir. Kullanıcı bu kodları göremez çünkü eğer sunucu tarafı kodları görünür olsaydı veritabanı bağlantıları, sistem mantığı ve güvenlik mekanizmaları açığa çıkabilirdi. Bu durum ciddi güvenlik açıklarına yol açabilir.

Command Injection (Komut Enjeksiyonu): Command Injection, bir web uygulamasında kullanıcıdan alınan verilerin yeterince kontrol edilmemesi sonucu saldırganın sunucu üzerinde sistem komutları çalıştırabilmesine neden olan bir güvenlik açığıdır. Örneğin bir uygulama kullanıcıdan alınan veriyi arka planda bir Linux komutuna ekleyerek çalıştırıyorsa, saldırgan bu alana ek komutlar yazabilir. Bu durumda saldırgan ls veya whoami gibi komutlar çalıştırarak sistem hakkında bilgi toplayabilir. Bu nedenle kullanıcı girdilerinin kontrol edilmesi ve doğrudan sistem komutlarına eklenmemesi gerekir.

3. Ağ Anatomisi: "Avin İzleri"

C2 (Command and Control): C2, bir sisteme bulaşan zararlı yazılımın (Malware) saldırgan ile iletişim kurduğu kontrol sunucusudur. Güvenlik duvarları (Firewall) genellikle dışarıdan gelen bağlantıları engellediği için zararlı yazılım çoğu zaman içeriden dışarıya doğru bir bağlantı kurar. Bu sayede saldırgan doğrudan sisteme bağlanmadan uzaktan komut gönderebilir. Zararlı yazılım belirli aralıklarla C2 sunucusuna bağlanarak yeni bir komut olup olmadığını kontrol eder. Bu iletişim sayesinde saldırgan sistemden veri toplayabilir veya çalınan bilgileri dışarı aktarabilir. Bu nedenle ağ analizlerinde C2 trafiğinin tespit edilmesi oldukça önemlidir.

PCAP Analizi: Antivirüs her zaman tüm tehditleri tespit edemeyebilir. Bazı zararlı yazılımlar antivirüsü atlatabilir veya sistemde iz bırakmadan çalışabilir. Bu yüzden analistler ağ trafiğini incelemek için PCAP dosyalarını analiz eder. PCAP dosyaları ağdaki tüm paketleri içerdiği için şüpheli IP adresleri, zararlı bağlantılar veya veri sızıntısı gibi durumlar daha kolay tespit edilebilir. Bu sayede zararlı yazılımın ağ üzerindeki davranışı daha net bir şekilde anlaşılabilir.

BÖLÜM B

Cephe 1: İşletim Sistemleri Ustası (Linux / Bash)

Level 10 → 11

Adım 1: Sistem üzerinde bulunan data.txt dosyası incelenir.

Adım 2: Dosya içeriği görüntülendiğinde metnin Base64 formatında kodlanmış olduğu fark edilir.

Adım 3: Bu veriyi çözmek için terminalde aşağıdaki komut çalıştırılır.

```
- base64 -d data.txt -
```

Adım 4: Komut çalıştırıldıktan sonra kodlanmış veri çözülür ve ekranda gerçek metin görüntülenir.

Adım 5: Çözülen metin içerisinde Bandit Level 11 parolası elde edilir.

```
kali@kali: ~
Session Actions Edit View Help
data.txt
bandit10@bandit:~$ whoami
bandit10
bandit10@bandit:~$ base64 -d data.txt
The password is dtR173fZKb0RRsDFSGsg2RWnpNVj3qRr
bandit10@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

(kali@kali)-[~]
$ ssh bandit11@bandit.labs.overthewire.org -p 2220

[OVER]

This is an OverTheWire game server.
More information on http://www.overthewire.org/wargames

backend: gibson-0
bandit11@bandit.labs.overthewire.org's password: 
```

Level 11 → 12

Adım 1: Bu seviyede verilen data.txt dosyasının hex dump formatında olduğu tespit edilir.

Adım 2: Çalışma alanı oluşturmak için /tmp dizini içerisinde yeni bir klasör oluşturulur.

Adım 3: data.txt dosyası bu klasöre kopyalanır.

Adım 4: Hex dump verisini tekrar binary formata çevirmek için aşağıdaki komut kullanılır.

- xxd -r data.txt > file -

Adım 5: Elde edilen metnin ROT13 şifreleme yöntemi ile kodlandığı fark edilir.

Adım 6: ROT13 dönüşümü uygulamak için aşağıdaki komut çalıştırılır.

- cat file | tr 'A-Za-z' 'N-ZA-Mn-za-m' -

Adım 7: Komut sonucunda elde edilen metin içerisinde Bandit Level 12 parolası bulunur.

```
bandit11@bandit: /tmp/bandit12_bandit11
Session Actions Edit View Help
bandit11@bandit:~$ whoami
bandit11
bandit11@bandit:~$ ls
data.txt
bandit11@bandit:~$ mkdir /tmp/bandit12
mkdir: cannot create directory '/tmp/bandit12': File exists
bandit11@bandit:~$ cd /tmp/bandit12
-bash: cd: /tmp/bandit12: Not a directory
bandit11@bandit:~$ mkdir /tmp/b12
cd /tmp/b12
mkdir: cannot create directory '/tmp/b12': File exists
bandit11@bandit: /tmp/b12$ cp ~/data.txt .
cp: cannot create regular file './data.txt': Permission denied
bandit11@bandit: /tmp/b12$ mkdir /tmp/bandit12_$USER
cd /tmp/bandit12_$USER
bandit11@bandit: /tmp/bandit12_bandit11$ cp ~/data.txt .
bandit11@bandit: /tmp/bandit12_bandit11$ ls
data.txt
bandit11@bandit: /tmp/bandit12_bandit11$ xxd -r data.txt > data
bandit11@bandit: /tmp/bandit12_bandit11$ cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
The password is 7x16WNeH1i5YkIhWsfFIqoognUTy9Q4
bandit11@bandit: /tmp/bandit12_bandit11$

bandit12@bandit: ~
--[ Tools ]--
For your convenience we have installed a few useful tools which you can find
in the following locations:
* gef (https://github.com/hugsy/gef) in /opt/gef/
* pwndbg (https://github.com/pwndbg/pwndbg) in /opt/pwndbg/
* gdbinit (https://github.com/gdbinit/gdbinit) in /opt/gdbinit/
* pwntools (https://github.com/Gallopsled/pwntools)
* radare2 (http://www.radare.org/)

--[ More information ]--
For more information regarding individual wargames, visit
http://www.overthewire.org/wargames/

For support, questions or comments, contact us on discord or IRC.
Enjoy your stay!
bandit12@bandit:~$ 
```

Level 12 → 13

Adım 1: Sistem üzerinde bulunan data.txt dosyasının hex dump formatında olduğu tespit edilir.

Adım 2: Çalışma sırasında sistem dosyalarına zarar vermemek için /tmp dizini içerisinde yeni bir çalışma klasörü oluşturulur.

- mkdir /tmp/bandit12 -

Adım 3: Oluşturulan klasöre geçilir.

- cd /tmp/bandit12 -

Adım 4: Çalışılacak olan data.txt dosyası bu dizine kopyalanır.

cp ~/data.txt .

Adım 5: Hex dump formatındaki veri tekrar binary formata çevrilir.

- xxd -r data.txt > file -

Adım 6: Elde edilen dosyanın türünü belirlemek için aşağıdaki komut kullanılır.

- file file -

Adım 7: Yapılan inceleme sonucunda dosyanın birden fazla sıkıştırma katmanı içerdiği görülür.

Adım 8: Dosya türüne göre sıkıştırma katmanları sırasıyla açılır.

Örnek olarak:

- mv file file.gz -

- gunzip file.gz -

- mv file file.bz2 -

- bunzip2 file.bz2 -

- tar -xf file.tar -

Adım 9: Tüm sıkıştırma katmanları açıldıktan sonra elde edilen dosya incelenir.

- cat data -

Adım 10: Dosya içeriğinde Bandit Level 13 parolası görüntülenir.

```
bandit12@bandit: /tmp/bandit12_bandit12
Session Actions Edit View Help

bandit12@bandit:~$ mkdir /tmp/bandit12_$USER
bandit12@bandit:~$ cd /tmp/bandit12_$USER
bandit12@bandit:/tmp/bandit12_$USER$ cp ~/data.txt .
bandit12@bandit:/tmp/bandit12_$USER$ xxd -r data.txt > data
bandit12@bandit:/tmp/bandit12_$USER$ cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
00000000: 1s80 0808 2817 fr68 0203 6461 7461 322r ...(.u..qngn2.
00000010: 6269 6r00 013p 02p3 sq42 5n68 3931 4159 ova.<...OMu91NL
00000020: 2653 59pp 4605 2q00 0818 ssss qn55 r6r3 8FL.S.....
00000030: 93p8 359q 0u69 0uq7 37s8 r7r7 0uq9 059s .....V.....
00000040: sss7 pssq sss8 000s sss8 s5sr 000i 3058 .....?..iK
00000050: 2406 8000 00q0 6834 6234 q000 6869 9000 $....u404..uv..
00000060: 1n7n 8003 40q0 01n1 n006 8188 340q 1n68 .m..@.....4..u
00000070: q340 q189 r906 8541 0346 4094 40q1 91n0 .@....N.SZ.@...
00000080: 681n 0681 n068 0660 p400 3207 r269 n189 u....u...2..V..
00000090: n326 8000 p800 p81n 1883 1000 00q0 0023 8.....#
00000100: 4311 n034 30pn 6800 0680 0681 n680 6868 P..40.u.....uu
00000110: q068 6868 p04p q400 0003 40q6 87n8 q000 .uuu.Y....Z....
00000120: 3086 8p20 3268 068q 000p 9n64 0698 8q04 0..2u....q....
00000130: 0600 0600 3541 2p03 p8r1 70pp 479r 2369 ..u5N...{.T..v
00000140: 30n1 0250 82r9 64r5 9040 3125 40p8 0005 0..C..q..01/X...
00000150: 0p8s 026p q5pn 1008 q7nn 336n rq8s 0070 ...Y.....3w...{

bandit12@bandit: /tmp/b12_bandit12
Session Actions Edit View Help

For support, questions or comments, contact us on discord or IRC.

Enjoy your stay!

bandit12@bandit:~$ mkdir /tmp/b12_$USER
cd /tmp/b12_$USER
cp ~/data.txt .
bandit12@bandit:/tmp/b12_$USER$ xxd -r data.txt > data
file data
data: gzip compressed data, was "data2.bin", last modified: Tue Oct 14 09:2
6:00 2025, max compression, from Unix, original size modulo 2^32 572
bandit12@bandit:/tmp/b12_$USER$ mv data data.gz
gunzip data.gz
file data
data: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/b12_$USER$ mv data data.bz2
bunzip2 data.bz2
file data
data: gzip compressed data, was "data4.bin", last modified: Tue Oct 14 09:2
6:00 2025, max compression, from Unix, original size modulo 2^32 20480
bandit12@bandit:/tmp/b12_$USER$ mv data data.gz
gunzip data.gz
file data

bandit12@bandit: /tmp/b12_bandit12
Session Actions Edit View Help

gunzip data.gz
file data
data: POSIX tar archive (GNU)
bandit12@bandit:/tmp/b12_bandit12$ mv data data.tar
tar -xvf data.tar
ls
data5.bin data5.tar data.txt
bandit12@bandit:/tmp/b12_bandit12$ file data5.bin
data5.bin: POSIX tar archive (GNU)
bandit12@bandit:/tmp/b12_bandit12$ tar -xvf data5.bin
ls
data5.bin data6.bin data5.tar data.txt
bandit12@bandit:/tmp/b12_bandit12$ file data6.bin
data6.bin: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/b12_bandit12$ mv data6.bin data6.bz2
bunzip2 data6.bz2
ls
data5.bin data6 data5.tar data.txt
bandit12@bandit:/tmp/b12_bandit12$ file data6file data6
data6file: cannot open 'data6file' (No such file or directory)
data6: POSIX tar archive (GNU)
bandit12@bandit:/tmp/b12_bandit12$ mv data6 data6.tar
tar -xvf data6.tar
ls
data5.bin data6.tar data8.bin data5.tar data.txt
bandit12@bandit:/tmp/b12_bandit12$ file data8.bin
data8.bin: gzip compressed data, was "data9.bin", last modified: Tue Oct 14
09:26:00 2025, max compression, from Unix, original size modulo 2^32 49
bandit12@bandit:/tmp/b12_bandit12$ mv data8.bin data8.gz
gunzip data8.gz
ls
data5.bin data6.tar data8 data5.tar data.txt
bandit12@bandit:/tmp/b12_bandit12$ cat data8
The password is F05dWfsc0cbalIH0H8J2eUks2vdTDwAn
bandit12@bandit:/tmp/b12_bandit12$
```

Level 13 → 14

Adım 1: Bu seviyede parola yerine SSH Private Key kullanılması gerektiği belirtilmektedir.

Adım 2: Kullanıcı dizininde bulunan sshkey.private dosyası incelenir.

Adım 3: Private key içeriği yeni bir dosyaya kaydedilir.

Adım 4: Anahtar dosyasının güvenliğini sağlamak için aşağıdaki komut kullanılır.

- chmod 600 key -

Adım 5: SSH bağlantısı aşağıdaki komut ile gerçekleştirilir.

- ssh -i key bandit14@localhost -

Adım 6: Sisteme giriş yaptıktan sonra aşağıdaki dosya okunur.

- cat /etc/bandit_pass/bandit14 -

Adım 7: Bu işlem sonucunda Bandit Level 14 parolası elde edilir.

```
bandit13@bandit:~$ cat sshkey.private
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAxkkOE83W2c0T7IWhFc9aPaaQmQDgZuXcv+ppZHa++buSkN+
gg0tcr7Fw8NLGa5+U2ec2rEg0WmeevB13AI0p0M2yEtq46t+jk9puNwZi19XG8
ZuF6L2EwmbPw/vLmW0Be4uS1GRWzGpPe5v5t1Vj1Z1Bd0pH1K22Am6Zb
TmWsiMwyJaFwJ/78PQ03m5yS91vUHEu0MA2oIUDakNM0E23+XahYK08JvQ68K5V
ObefXG1vA3GA29Kx3JaqrFgYnqZrYWN7w3CHjNU4c/23Kp+n8L0SnxaNA+WYA7
jiPyTF0is8uzMlyQ411Lzh/8/MpvhCQFr22dwIDAQABAQIBAQc6dWbJhyE0zjeA
J3j/Rwmap9M5fzj/wb2bfidNpWb8rs34s2IDZQ7Xuih4LfgoAQSS+bW3RXvze
pV3L3SmJsh1DulscJL1vMB5p7BjudeGaR/3fYjYNAqZ/TLFzLLYFOU719Jete67
xAh0tONG/uRFB513LA12Vp6Dv1wdw6cAn0aCt1h1dpuPKLABrMMVRTK0+7T2VS
nXmWYcKkUcUgzo5p1NzaS0zUDydpdy2+trH3Mq5a5kN1YKjvF8RC47woDYCKtsD
o3FFpGfnc9Taa3M5y+DFQqHkZFK13JD0NtmrVvTYK40/yelU4aZ/HA2DQzwe
o1AFiEhA0GBA0nVjos8km7ablk+n4IEwPxS850mhPnTDUy5WGrpScRcX0msVIBUF
la13ZGL3xCIwTcnEucB9DvN2HZkuPC/h6hTKUyLqXuyLD8n3TrbRhlgBC9QrKrS
M1F2f5tXvqPtZ0LDmWjNR04xHA/fKh8bXxyTMqOHUJTHHhnh3McduRjA0GBANKU
1hqfW7+aXnc39bJysr12WbqOE5Nd8AFgFwaKUGTTVX2NsUqNCMD0p+wFak403H
-----END RSA PRIVATE KEY-----
bandit13@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

```
bandit14@bandit:~$ cat /etc/bandit_pass/bandit14
MU4VWeTyJk8R0of1qqmcBPALh7LDCPV5
bandit14@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

(kali@kali)~$ nano bandit14.key
(kali@kali)~$ chmod 600 bandit14.key
(kali@kali)~$ ssh -i bandit14.key bandit14@bandit.labs.overthewire.org -p 2220

[bandit14@bandit:~$ cat /etc/bandit_pass/bandit14
MU4VWeTyJk8R0of1qqmcBPALh7LDCPV5
bandit14@bandit:~$
```

Level 14 → 15

Adım 1: Bu seviyede mevcut parolanın localhost üzerinde çalışan bir servise gönderilmesi gerekmektedir.

Adım 2: Servisin 30000 portu üzerinde çalıştığı tespit edilir.

Adım 3: Bu porta bağlanmak için netcat (nc) aracı kullanılır.

- nc localhost 30000 -

Adım 4: Açılan bağlantı üzerinden mevcut parola gönderilir.

Adım 5: Sistem parolayı doğruladıktan sonra ekranda Bandit Level 15 parolasını görüntüler.

```
bandit14@bandit:~$ nc localhost 30000
MU4VWeTyJk8R0of1qqmcBPALh7LDCPV5
Correct!
8xCjnmgoKbGLHFAZlG5Tmu4M2tKJQo

bandit14@bandit:~$
```

Level 15 → 16

Adım 1: Bu seviyede mevcut parolanın SSL ile şifrelenmiş bir servise gönderilmesi gerektiği belirtilmektedir.

Adım 2: Servisin localhost üzerinde 30001 portunda çalıştığı tespit edilir.

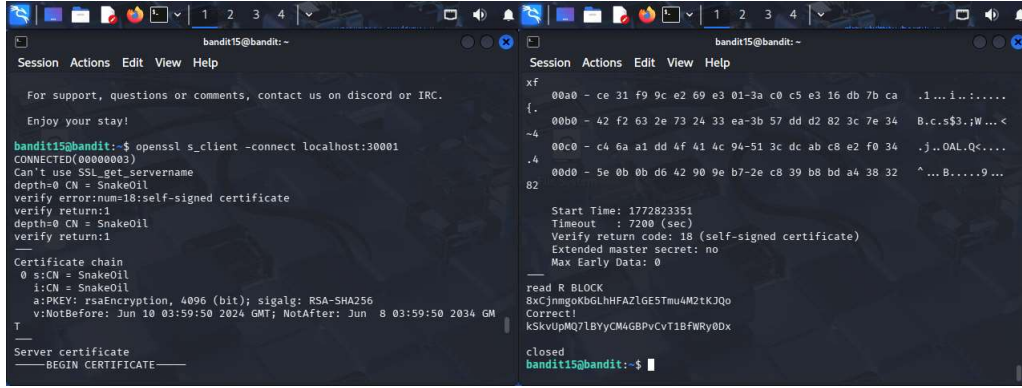
Adım 3: SSL bağlantısı kurabilmek için openssl s_client aracı kullanılır.

- openssl s_client -connect localhost:30001 -

Adım 4: Bağlantı kurulduktan sonra sistem tarafından bir SSL oturumu başlatılır.

Adım 5: Açılan bağlantı üzerinden mevcut Bandit15 parolası gönderilir.

Adım 6: Sistem parolayı doğruladıktan sonra ekranda Bandit Level 16 parolası görüntülenir.



```
bandit15@bandit:~$ openssl s_client -connect localhost:30001
CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 CN = SnakeOil
verify error:num=18:self-signed certificate
verify return:1
depth=0 CN = SnakeOil
verify return:1
---
Certificate chain
 0 s:CN = SnakeOil
  i:CN = SnakeOil
  a:PKEY: rsaEncryption, 4096 (bit); sigalg: RSA-SHA256
  v:NotBefore: Jun 10 03:59:50 2024 GMT; NotAfter: Jun  8 03:59:50 2034 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----

```

```
bandit15@bandit:~$
xf
00a0 - ce 31 f9 9c e2 69 e3 01-3a c0 c5 e3 16 db 7b ca .1...i.....
{.
00b0 - 42 f2 63 2e 73 24 33 ea-3b 57 dd d2 82 3c 7e 34 B.c.s$3.iW...<
~4
00c0 - c4 6a a1 dd 4f 41 4c 94-51 3c dc ab c8 e2 f0 34 .j..OAL.Q<....
.4
00d0 - 5e 0b 0b d6 42 90 9e b7-2e c8 39 b8 bd a4 38 32 ^...B.....9...
82

Start Time: 1772823351
Timeout : 7200 (sec)
Verify return code: 18 (self-signed certificate)
Extended master Secret: no
Max Early Data: 0

read R BLOCK
8xCjnmgoKbGLHFAZLGE5Tmu4M2LKJQo
Correct!
kSkvUpMQ7lBYyCMAGBPvCvT1BFwRy0Dx

closed
bandit15@bandit:~$
```

Cephe 2: Web Mantık Avcısı (Server-Side)

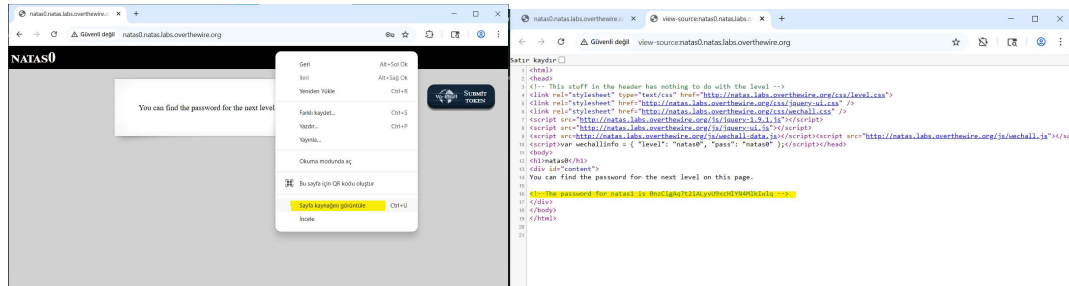
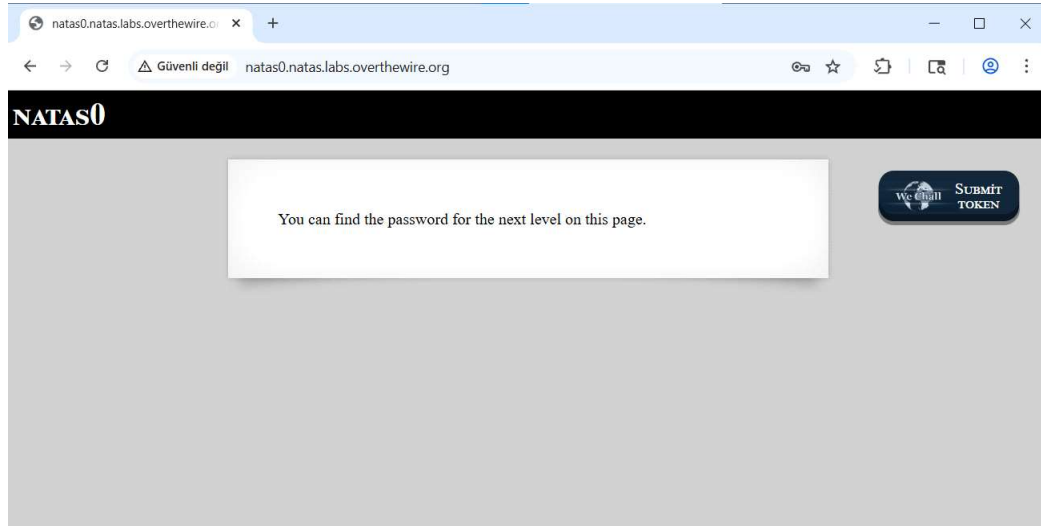
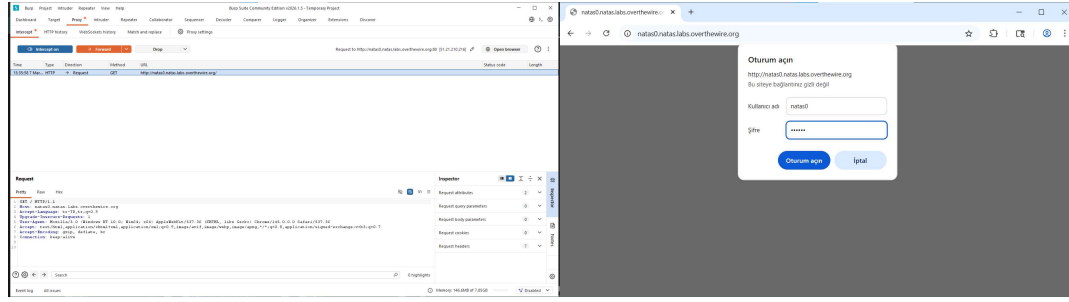
Natas0 → Natas1

Adım 1: Natas0 web sayfası tarayıcıda açılır.

Adım 2: Sayfa üzerinde görünür bir parola bulunmadığı için sayfanın kaynak kodu incelenir.

Adım 3: Kaynak kodunu görüntülemek için tarayıcıda View Source veya Ctrl + U kısayolu kullanılır.

Adım 4: HTML yorum satırları incelendiğinde bir sonraki seviyeye ait Natas1 parolası bulunur.



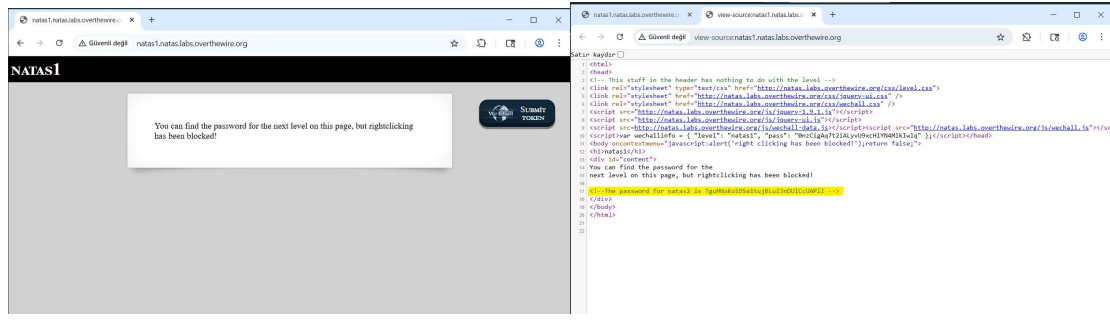
Natas1 → Natas2

Adım 1: Sayfada sağ tıklamanın engellendiği görülür.

Adım 2: Bu engeli aşmak için tarayıcıda Ctrl + U kısayolu kullanılarak kaynak kodu açılır.

Adım 3: Kaynak kodu içerisinde bulunan yorum satırları incelenir.

Adım 4: Yorum satırları içerisinde gizlenmiş olan Natas2 parolası elde edilir.



Natas2 → Natas3

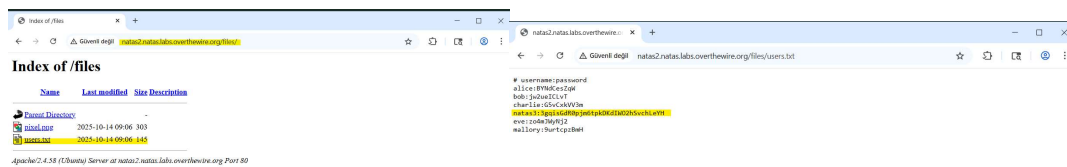
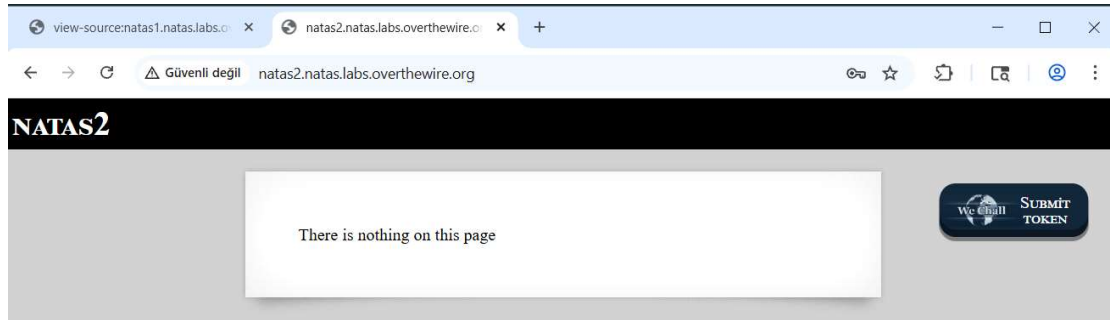
Adım 1: Sayfa üzerinde herhangi bir ipucu bulunmadığı için kaynak kodu incelenir.

Adım 2: Kaynak kodu içerisinde files isimli bir dizine yönlendiren bağlantı fark edilir.

Adım 3: Tarayıcı üzerinden bu dizine gidilir.

Adım 4: Dizinde bulunan users.txt dosyası açılır.

Adım 5: Dosya içerisinde Natas3 parolası elde edilir.



Natas3 → Natas4

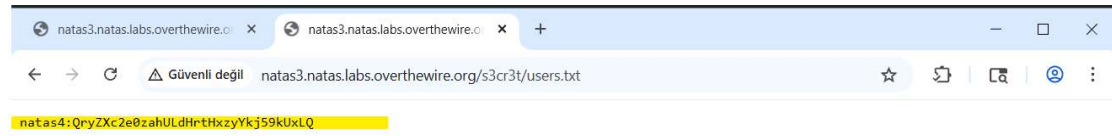
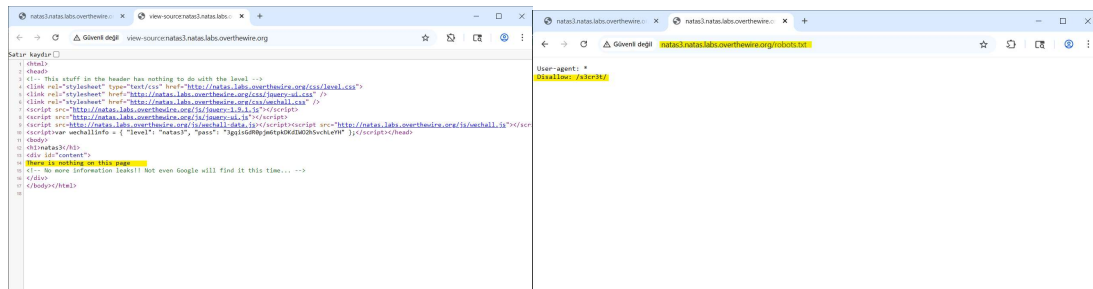
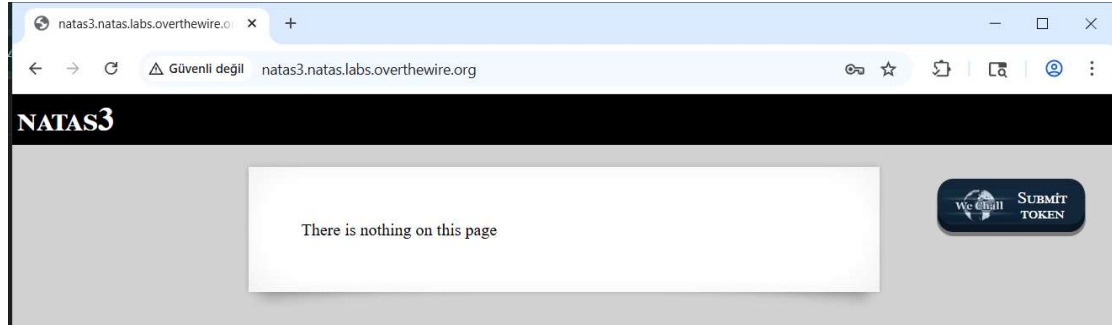
Adım 1: Sayfa ve kaynak kodu incelenir.

Adım 2: Kaynak kodu içerisinde robots.txt dosyasına yönlendiren bir ipucu olduğu fark edilir.

Adım 3: Tarayıcı üzerinden robots.txt dosyası açılır.

Adım 4: Bu dosya içerisinde arama motorlarından gizlenmiş olan /s3cr3t/ dizini tespit edilir.

Adım 5: Bu dizin içerisine gidildiğinde bulunan users.txt dosyasından Natas4 parolası elde edilir.



Natas4 → Natas5

Adım 1: Sayfanın yalnızca belirli bir adresten gelen isteklere izin verdiği görülür.

Adım 2: HTTP isteği Burp Suite Proxy kullanılarak yakalanır.

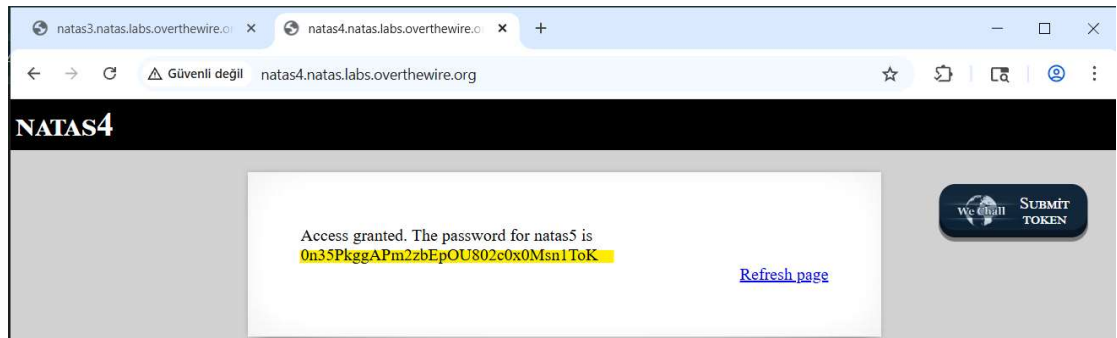
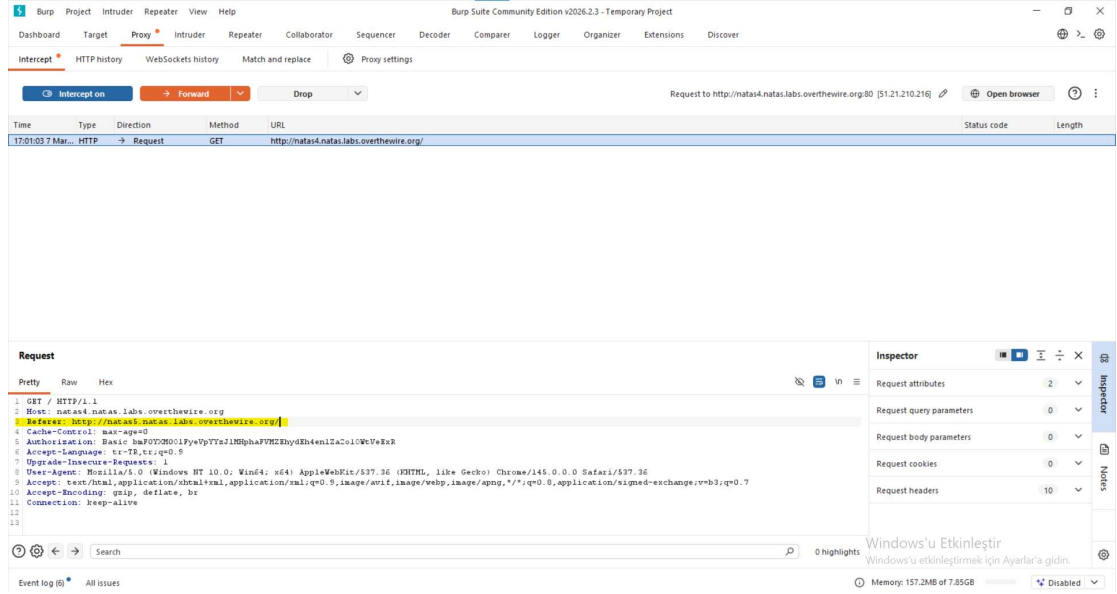
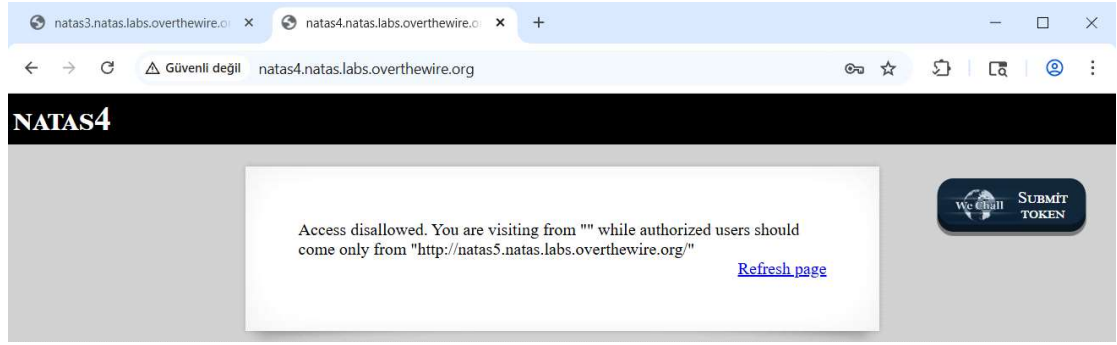
Adım 3: İstek içerisinde bulunan Referer başlığı incelenir.

Adım 4: Referer değeri aşağıdaki şekilde değiştirilir.

Referer: <http://natas5.natas.labs.overthewire.org/>

Adım 5: Değiştirilen istek tekrar gönderilir.

Adım 6: Sunucu isteği kabul eder ve ekranda Natas5 parolası görüntülenir.



Natas5 → Natas6

Adım 1: Sayfa açıldığında kullanıcının giriş yapmadığını belirten bir mesaj görüntülenir.

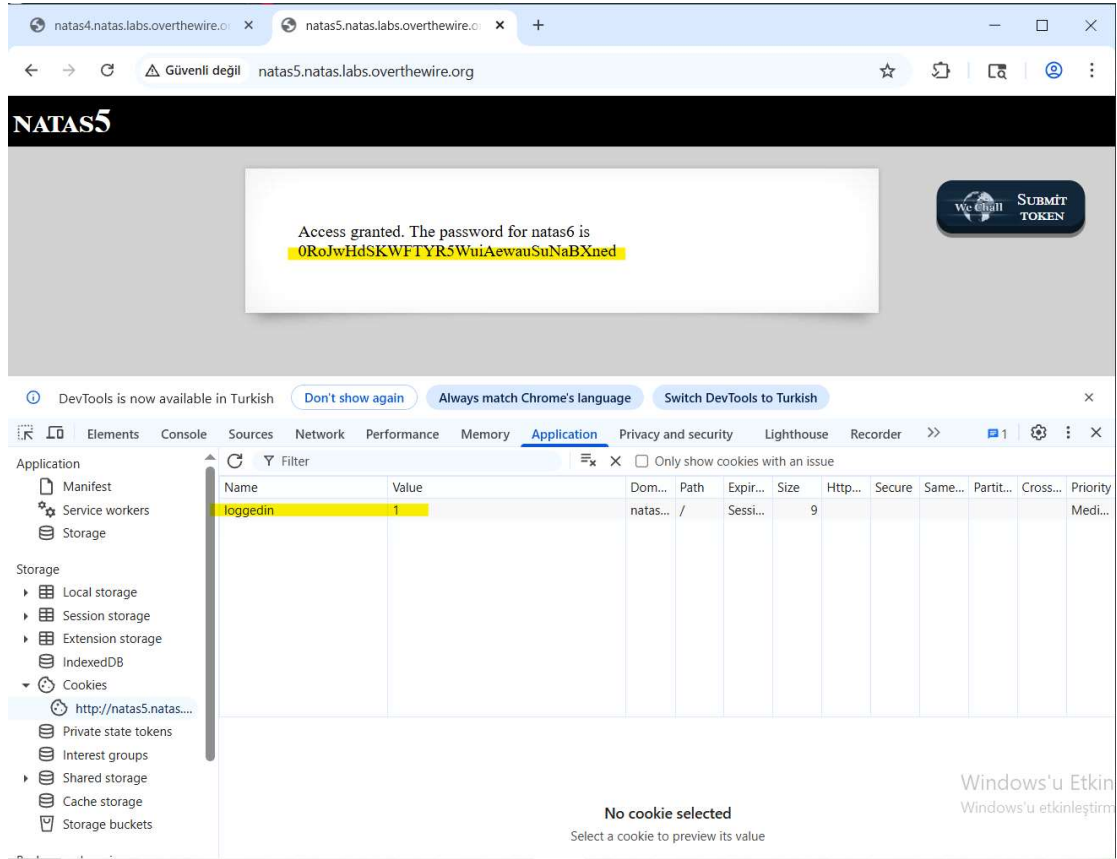
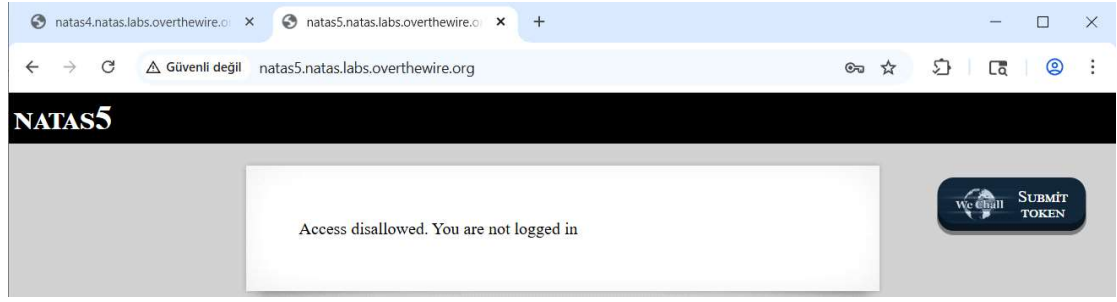
Adım 2: Tarayıcı geliştirici araçları kullanılarak Cookie değerleri incelenir.

Adım 3: loggedin değerinin 0 olduğu görülür.

Adım 4: Bu değer manuel olarak 1 olarak değiştirilir.

Adım 5: Sayfa yenilendiğinde sistem kullanıcıyı giriş yapmış olarak kabul eder.

Adım 6: Bu işlem sonucunda Natas6 parolası elde edilir.



```
natas5.natas.labs.overthe x natas6.natas.labs.overthe x view-source:natas6.natas x natas6.natas.labs.overthe x + - □ ×
← → ↻ Güvenli değil natas6.natas.labs.overthewire.org/index-source.html ☆ 📄 🔍 🌐 ⋮

<html>
<head>
<!-- This stuff in the header has nothing to do with the level -->
<link rel="stylesheet" type="text/css" href="http://natas.labs.overthewire.org/css/level.css">
<link rel="stylesheet" href="http://natas.labs.overthewire.org/css/jquery-ui.css" />
<link rel="stylesheet" href="http://natas.labs.overthewire.org/css/wechall.css" />
<script src="http://natas.labs.overthewire.org/js/jquery-1.9.1.js"></script>
<script src="http://natas.labs.overthewire.org/js/jquery-ui.js"></script>
<script src="http://natas.labs.overthewire.org/js/wechall-data.js"></script><script src="http://natas.labs.overthewire.org/js/wechall.js">
</script>
<script>var wechallinfo = { "level": "natas6", "pass": "<censored>" };</script></head>
<body>
<h1>natas6</h1>
<div id="content">

<?
include "includes/secret.inc";

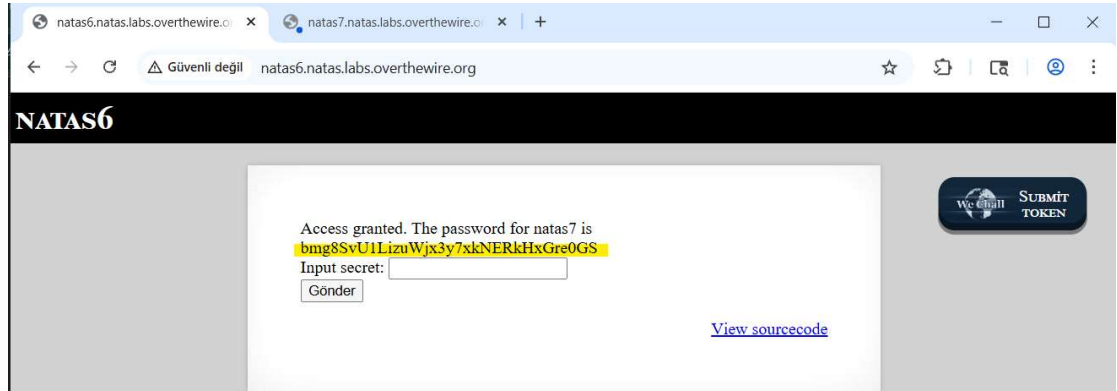
    if(array_key_exists("submit", $_POST)) {
        if($_secret == $_POST['secret']) {
            print "Access granted. The password for natas7 is <censored>";
        } else {
            print "Wrong secret";
        }
    }
?>

<form method=post>
Input secret: <input name=secret><br>
<input type=submit name=submit>
</form>

<div id="viewsource"><a href="index-source.html">View sourcecode</a></div>
</div>
</body>
</html>
```

```
natas5.natas.labs.overthe x natas6.natas.labs.overthe x view-source:natas6.natas x natas6.natas.labs.overthe x + - □ ×
← → ↻ Güvenli değil natas6.natas.labs.overthewire.org/includes/secret.inc ☆ 📄 🔍 🌐 ⋮

<?
$_secret = "FOEIUWGHFEEUHOFOUOIU";
?>
```



Natas6 → Natas7

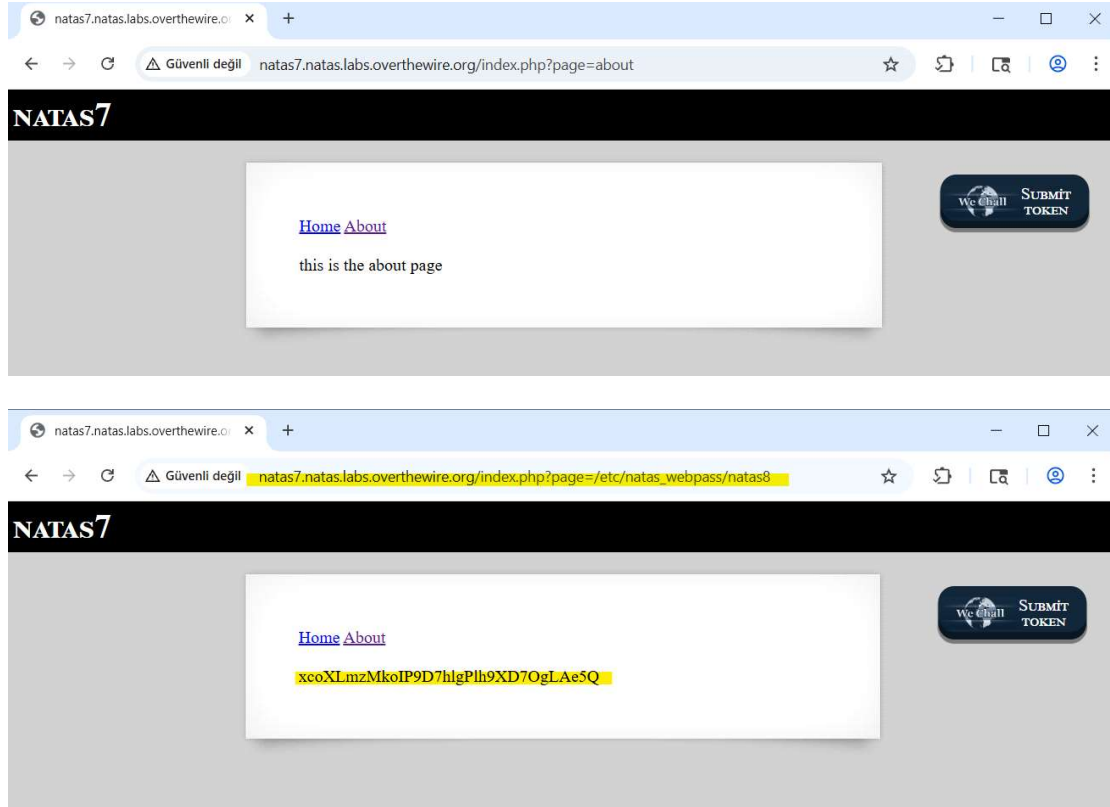
Adım 1: Sayfada bir secret değeri istenir.

Adım 2: Kaynak kodu incelendiğinde includes/secret.inc dosyasının çağrıldığı görülür.

Adım 3: Tarayıcı adres çubuğuna bu dosya yolu yazılarak dosyaya doğrudan erişilir.

Adım 4: Dosya içerisinde bulunan gizli kod elde edilir.

Adım 5: Bu kod form alanına girildiğinde Natas7 parolası elde edilir.



Natas7 → Natas8

Adım 1: Sayfa URL yapısında page parametresi kullanıldığı fark edilir.

Adım 2: Bu parametre üzerinden sistem dosyalarına erişim denenir.

Adım 3: URL aşağıdaki şekilde değiştirilir.

- /etc/natas_webpass/natas8 -

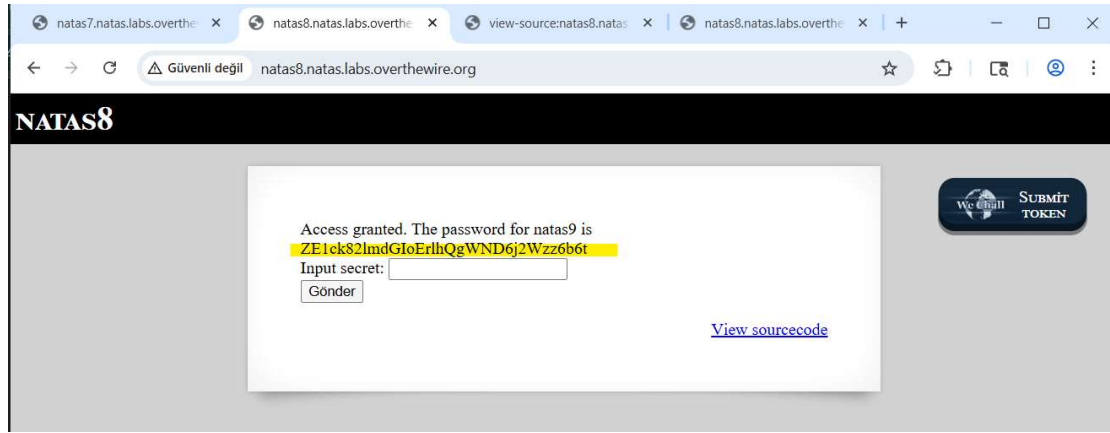
Adım 4: Local File Inclusion (LFI) zafiyetinden yararlanılarak sistem dosyası okunur.

Adım 5: Dosya içerisinde Natas8 parolası elde edilir.

```
<html>
<head>
<!-- This stuff in the header has nothing to do with the level -->
<link rel="stylesheet" type="text/css" href="http://natas.labs.overthewire.org/css/level.css">
<link rel="stylesheet" href="http://natas.labs.overthewire.org/css/jquery-ui.css" />
<link rel="stylesheet" href="http://natas.labs.overthewire.org/css/wechall.css" />
<script src="http://natas.labs.overthewire.org/js/jquery-1.9.1.js"></script>
<script src="http://natas.labs.overthewire.org/js/jquery-ui.js"></script>
<script src="http://natas.labs.overthewire.org/js/wechall-data.js"></script><script src="http://natas.labs.overthewire.org/js/wechall.js">
</script>
<script>var wechallinfo = { "level": "natas8", "pass": "<censored>" };</script></head>
<body>
<h1>natas8</h1>
<div id="content">

<?
EncodedSecret = "3d3d516343746d4d6d6c315669563362";

function encodeSecret($secret) {
    return bin2hex(strrev(base64_encode($secret)));
}
```



Natas8 → Natas9

Adım 1: Sayfada verilen encoded secret değeri incelenir.

Adım 2: Kaynak kodu analiz edilerek değerın aşağıdaki işlemlerden geçtiğı tespit edilir:

- base64

- hex

- string reverse

- **Adım 3:** Bu işlemler tersine uygulanarak gizli değeri çözölür.

Adım 4: Çözölün değeri forma girildiğinde Natas9 parolası elde edilir.

```
natas8.natas.labs.overthe... x natas9.natas.labs.overthe... x view-source:natas9.natas... x natas9.natas.labs.overthe... x + - □ ×
← → ↻ △ Güvenli değil natas9.natas.labs.overthewire.org/index-source.html ☆ 📄 🔍 ⓘ ⋮

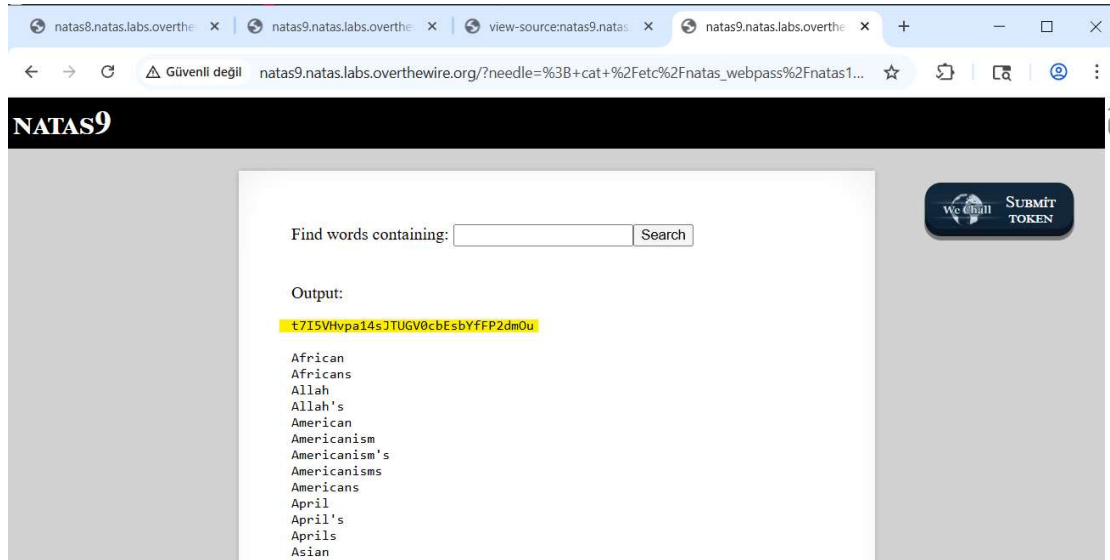
<html>
<head>
<!-- This stuff in the header has nothing to do with the level -->
<link rel="stylesheet" type="text/css" href="http://natas.labs.overthewire.org/css/level.css">
<link rel="stylesheet" href="http://natas.labs.overthewire.org/css/jquery-ui.css" />
<link rel="stylesheet" href="http://natas.labs.overthewire.org/css/wechall.css" />
<script src="http://natas.labs.overthewire.org/js/jquery-1.9.1.js"></script>
<script src="http://natas.labs.overthewire.org/js/jquery-ui.js"></script>
<script src="http://natas.labs.overthewire.org/js/wechall-data.js"></script><script src="http://natas.labs.overthewire.org/js/wechall.js">
</script>
<script>var wechallinfo = { "level": "natas9", "pass": "<censored>" };</script></head>
<body>
<h1>natas9</h1>
<div id="content">
<form>
Find words containing: <input name="needle"><input type="submit" name="submit" value="Search"><br><br>
</form>

Output:
<pre>
<?
$key = "";

if(array_key_exists("needle", $_REQUEST)) {
    $key = $_REQUEST["needle"];
}

if($key != "") {
    passthru("grep -i $key dictionary.txt");
}
?>
</pre>

<div id="viewsource"><a href="index-source.html">View sourcecode</a></div>
</div>
</body>
</html>
```



Natas9 → Natas10

Adım 1: Sayfadaki arama kutusunun arka planda grep komutunu çalıştırdığı fark edilir.

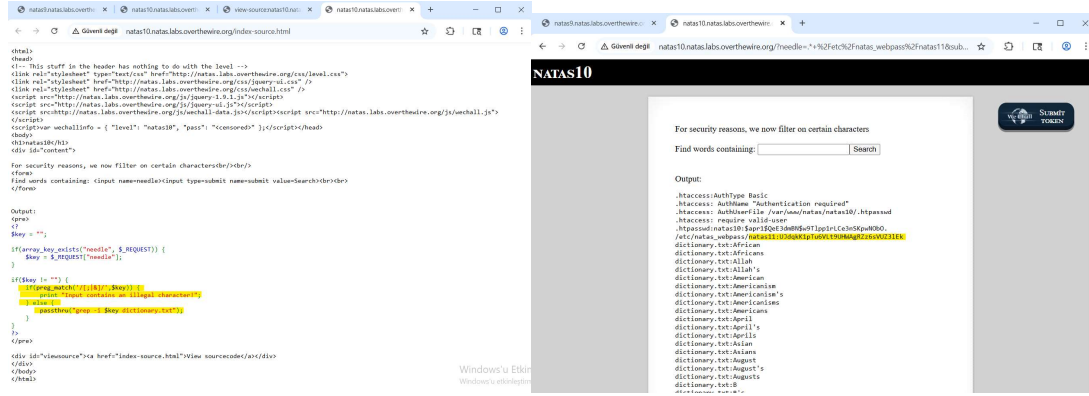
Adım 2: Kullanıcı giriş alanına ek komut yazılarak komut enjeksiyonu denenir.

Adım 3: Giriş alanına aşağıdaki ifade yazılır.

- ; cat /etc/natas_webpass/natas10 -

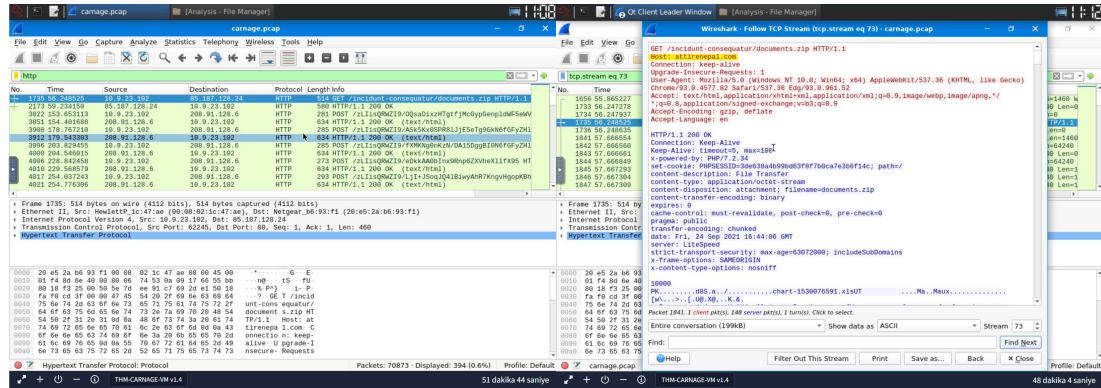
Adım 4: Sistem komutu çalıştırarak hedef dosyanın içeriğini ekrana yazdırır.

Adım 5: Bu işlem sonucunda Natas10 parolası elde edilir.

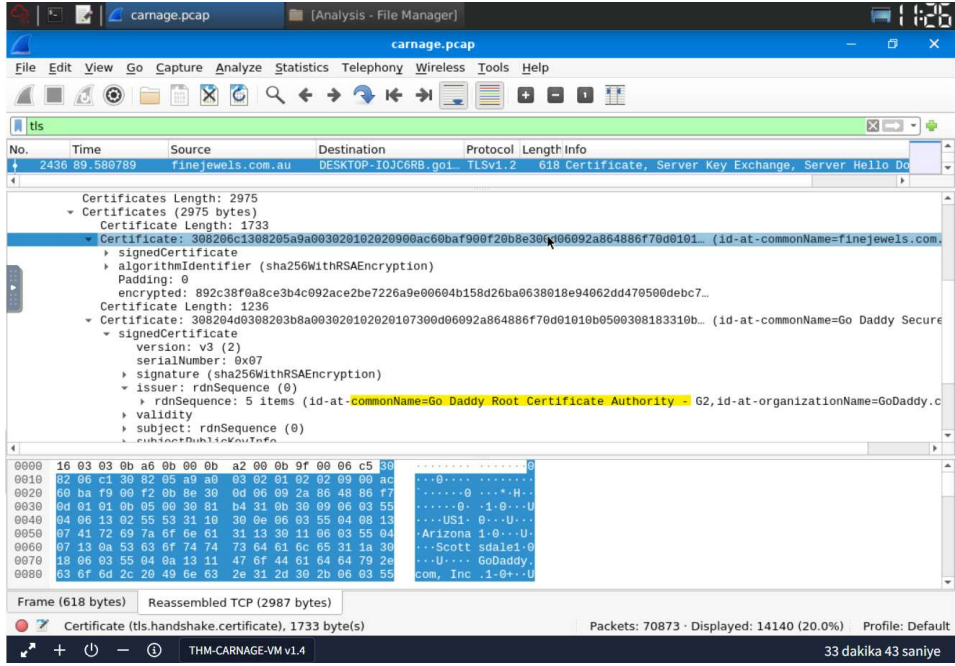


Cephe 3: Kör Nokta Dedektifi (Ağ Adli Analizi)

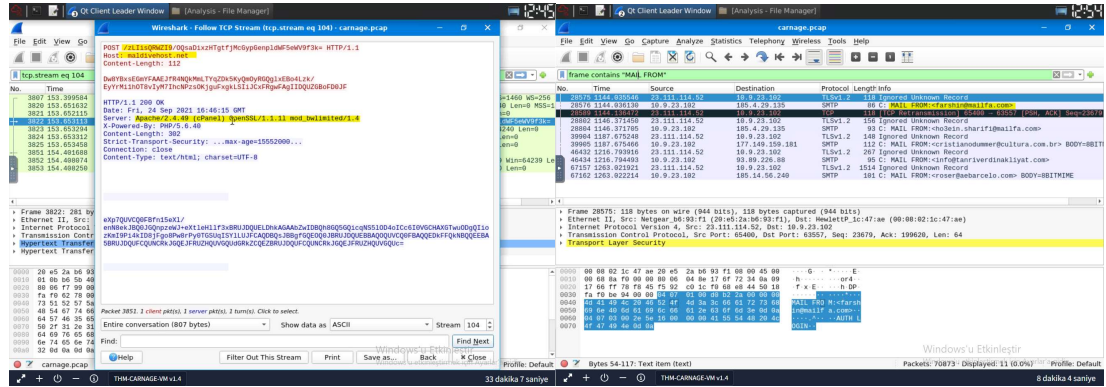
Ağ trafiği analizine başladığımda, ilk aşamada sistemdeki dosya transferlerini incelemek için `http.request.method == "GET"` filtresini kullandım. Yapılan inceleme sonucunda, kurban makinenin (10.9.23.102) harici bir adresten `documents.zip` isimli bir dosyayı indirdiğini tespit ettim. Bu transferin `attirenepal.com` alan adı üzerinden yapıldığı ve saldırının başlangıç noktası olduğu saptadım.



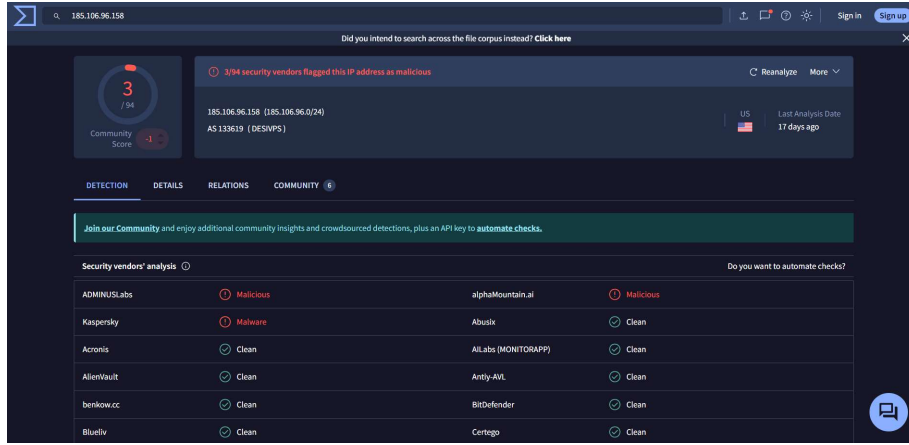
Saldırganın trafiği gizleme yöntemlerini anlamak için SSL sertifika detaylarına odaklandım. `tls.handshake.type == 11` filtresi ile yaptığım kontrolde, ağdaki şifreli trafiğin GoDaddy (Go Daddy Secure Certificate Authority) tarafından onaylanan bir sertifika ile yürütüldüğünü belirledim. Bu durum, saldırganın trafiği meşru göstermek için bir otoriteyi kullandığını kanıtlamış oldu.



Analizin devamında, kurban makinenin dış dünya ile kurduğu düzenli iletişimi (beaconing) ortaya çıkarmak amacıyla http.request.method == "POST" filtresini uyguladım. Bu sorgu sonucunda kurbanın, 185.106.96.158 IP adresine (maldivehost.net) sürekli veri gönderdiği açıkça gördüm. Ayrıca saldırganın, kurbanın dış IP adresini öğrenmek için api.ipify.org servisini kullandığı ve SMTP protokolü üzerinden e-posta trafiği (farshin@mailfa.com) yürüterek veri sızdırmaya çalıştığı tespit ettim.



Tespit edilen şüpheli IP adreslerini (özellikle 185.106.96.158) VirusTotal platformu üzerinden sorgulattım. Yapılan istihbarat çalışması sonucunda, bu adreslerin birden fazla güvenlik motoru tarafından "Malicious" (Zararlı) ve "Cobalt Strike C2" sunucusu olarak damgalandığı doğrulanmış oldu.



Indicators of Compromise (IOC)

Yapılan ağ trafiği analizi sonucunda saldırı ile ilişkili olduğu değerlendirilen bazı göstergeler (Indicators of Compromise – IOC) tespit edilmiştir. Bu göstergeler, benzer saldırıların tespit edilmesi ve güvenlik sistemlerinde (IDS/IPS, SIEM vb.) alarm oluşturulması için kullanılabilir.

IP Address: 185.106.96.158

Açıklama: Kurban makinenin POST istekleri gönderdiği ve beaconing davranışı sergileyen Cobalt Strike C2 sunucusu.

IP Address: 185.125.204.174

Açıklama: VirusTotal üzerinde Cobalt Strike C2 altyapısı ile ilişkilendirilen ikinci kontrol sunucusu.

Domain: maldivehost.net

Açıklama: Kurban makinenin zararlı iletişim kurduğu alan adı.

File: documents.zip

Açıklama: Kurban makine tarafından indirilen ve saldırının başlangıcında kullanılan şüpheli dosya.

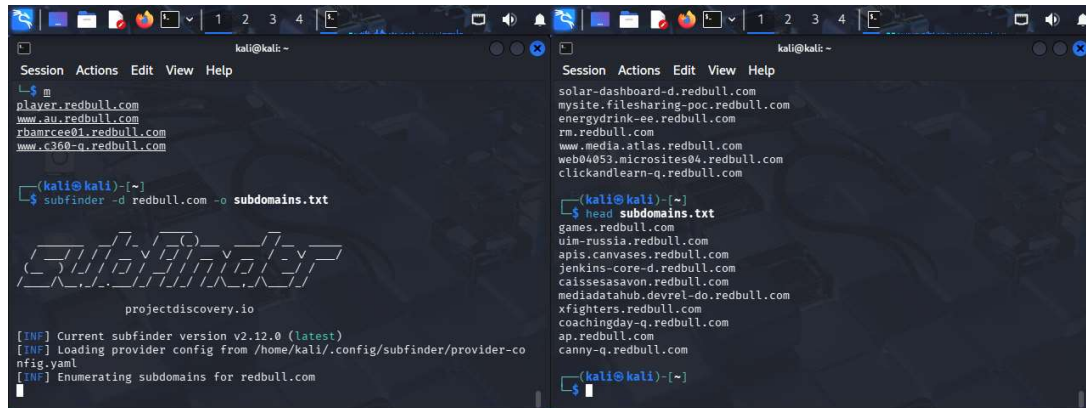
BÖLÜM C

Cephe 1: Hedef Seçimi ve Keşif (Recon Haritası)

Bu aşamada gerçek bir sızma testi simülasyonu yapmak için Bugcrowd üzerinde herkese açık ve test izni bulunan Red Bull firmasını hedef olarak seçtim. Amacım, ana sistem yerine daha az dikkat çeken alt sistemleri tespit ederek potansiyel zafiyet noktalarını belirlemektir.

Keşif sürecinin ilk adımında hedef domaine ait alt alan adlarını bulmak için subfinder aracını kullandım. Bunun için Kali Linux terminalinde subfinder -d redbull.com -o subdomains.txt komutunu çalıştırdım. Bu komut sayesinde Red Bull'a ait birçok subdomain tespit ettim ve sonuçlar subdomains.txt dosyasına kaydettim. Liste oldukça uzun olduğu için

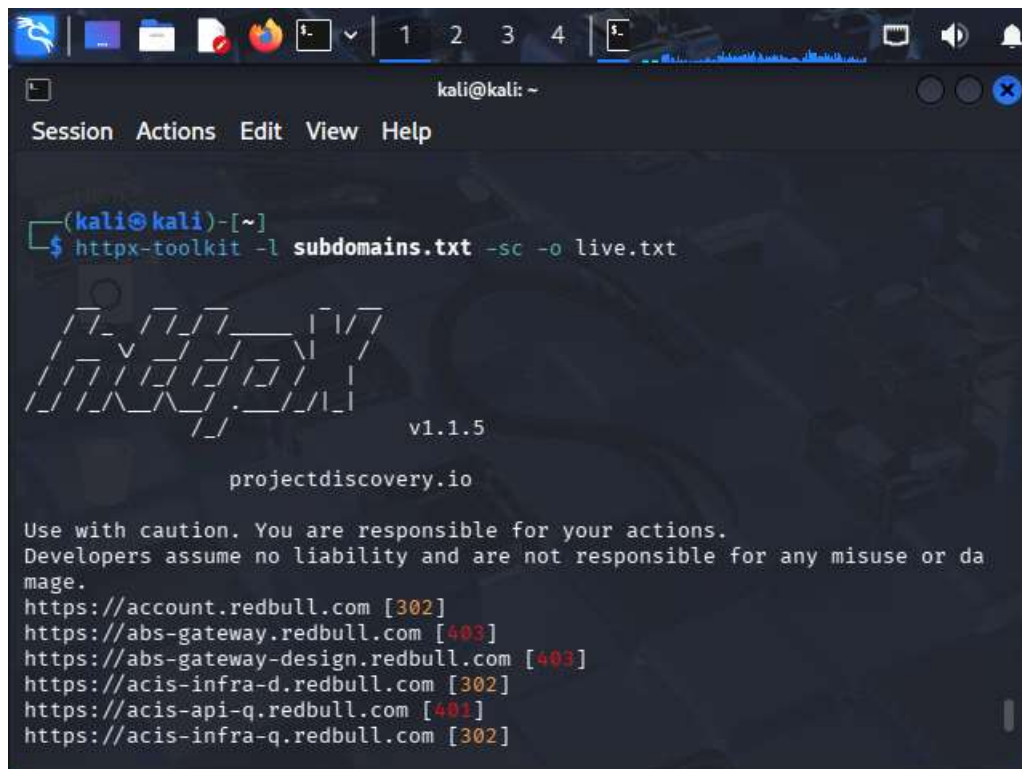
ilk sonuçları görmek amacıyla head subdomains.txt komutunu kullandım. Bu işlem sonucunda games.redbull.com, apis.canvases.redbull.com, jenkins-core-d.redbull.com gibi farklı alt alan adları elde etmiş oldum.



The image shows two terminal windows side-by-side. The left window shows the output of the 'subfinder' command, listing various subdomains of redbull.com. The right window shows the output of the 'head' command on the 'subdomains.txt' file, displaying a list of subdomains including games.redbull.com, apis.canvases.redbull.com, and jenkins-core-d.redbull.com.

```
kali@kali: ~  
$ subfinder -d redbull.com -o subdomains.txt  
  
projectdiscovery.io  
[INF] Current subfinder version v2.12.0 (latest)  
[INF] Loading provider config from /home/kali/.config/subfinder/provider-co  
nfig.yaml  
[INF] Enumerating subdomains for redbull.com  
  
solar-dashboard-d.redbull.com  
mysite.filesharing-poc.redbull.com  
energydrink-ee.redbull.com  
rm.redbull.com  
www.media.atlas.redbull.com  
web04053.micosites04.redbull.com  
clickandlearn-q.redbull.com  
  
$ head subdomains.txt  
games.redbull.com  
uin-russia.redbull.com  
apis.canvases.redbull.com  
jenkins-core-d.redbull.com  
caissesasavon.redbull.com  
mediadatahub.devrel-do.redbull.com  
xfighters.redbull.com  
coachingday-q.redbull.com  
ap.redbull.com  
canny-q.redbull.com
```

Subdomainleri bulduktan sonra bu adreslerin hangilerinin aktif olduğunu belirlemek için httpx aracını kullandım. Bunun için httpx-toolkit -l subdomains.txt -sc -o live.txt komutunu çalıştırdım. Bu işlem sonucunda bazı adreslerin 200, 302, 403 ve 401 gibi durum kodları döndürdüğü gördüm. Bu durum kodları sunucuların aktif olduğunu ancak bazı kaynaklara erişimin kısıtlanmış olabileceğini göstermektedir. Aktif olarak çalışan adreslerden biri <https://abp.redbull.com> [200] olarak tespit ettim.

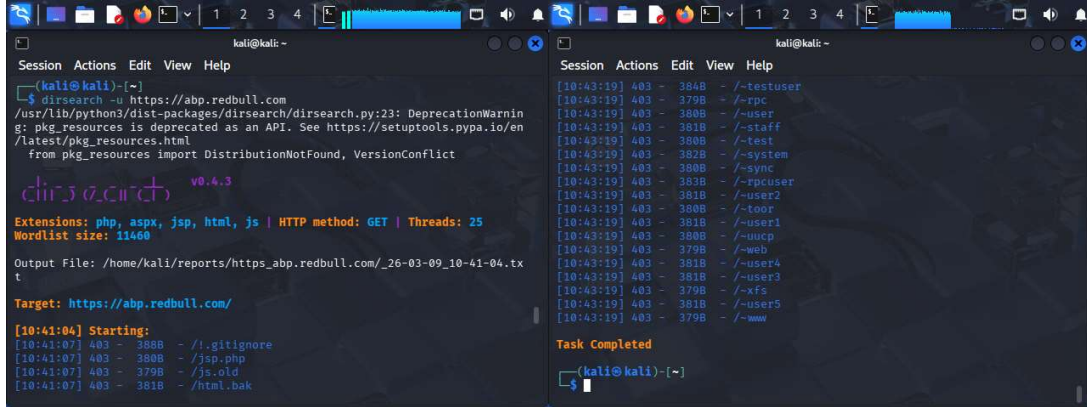


The image shows a terminal window with the output of the 'httpx-toolkit' command. It lists several subdomains and their corresponding HTTP status codes. The status codes are color-coded: 302 is green, 403 is red, and 401 is blue.

```
kali@kali: ~  
$ httpx-toolkit -l subdomains.txt -sc -o live.txt  
  
v1.1.5  
projectdiscovery.io  
  
Use with caution. You are responsible for your actions.  
Developers assume no liability and are not responsible for any misuse or da  
mage.  
https://account.redbull.com [302]  
https://abs-gateway.redbull.com [403]  
https://abs-gateway-design.redbull.com [403]  
https://acis-infra-d.redbull.com [302]  
https://acis-api-q.redbull.com [401]  
https://acis-infra-q.redbull.com [302]
```

Daha sonra aktif olan bu site üzerinde gizli dizinleri bulmak için dirsearch aracını kullandım. Bunun için dirsearch -u <https://abp.redbull.com> komutunu çalıştırdım. Dirsearch aracı, yaygın kullanılan dizin isimlerini test ederek sistem üzerinde erişilebilir veya gizli kalmış

dizinleri bulmaya çalışır. Tarama sonucunda bazı dizinlerin 403 Forbidden hatası verdiği gördüm. Bu da ilgili dizinlerin sunucu üzerinde bulunduğunu ancak erişimin engellendiğini gösterdi



```
Session Actions Edit View Help
(kali@kali) [~]
$ dirsearch -u https://abp.redbull.com
/usr/lib/python3/dist-packages/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25
Wordlist size: 11460

Output File: /home/kali/reports/https_abp.redbull.com/_26-03-09_10-41-04.txt

Target: https://abp.redbull.com/

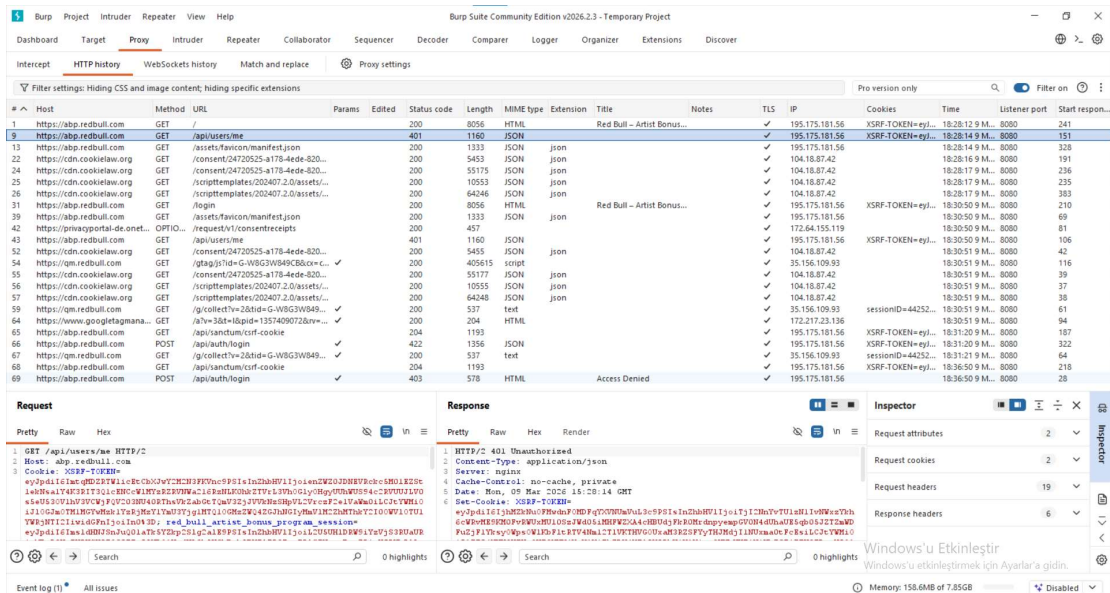
[10:41:04] Starting:
[10:41:07] 403 - 3080 - /!.gitignore
[10:41:07] 403 - 3080 - /jsp.php
[10:41:07] 403 - 3798 - /js.old
[10:41:07] 403 - 3818 - /html.bak

Task Completed
(kali@kali) [~]
$
```

Sonuç olarak yaptığım bu keşif çalışmasıyla Red Bull'a ait bazı alt alan adlarını, aktif çalışan sistemleri ve bu sistemlerin izin yapısı hakkında temel bilgileri elde etmiş oldum.

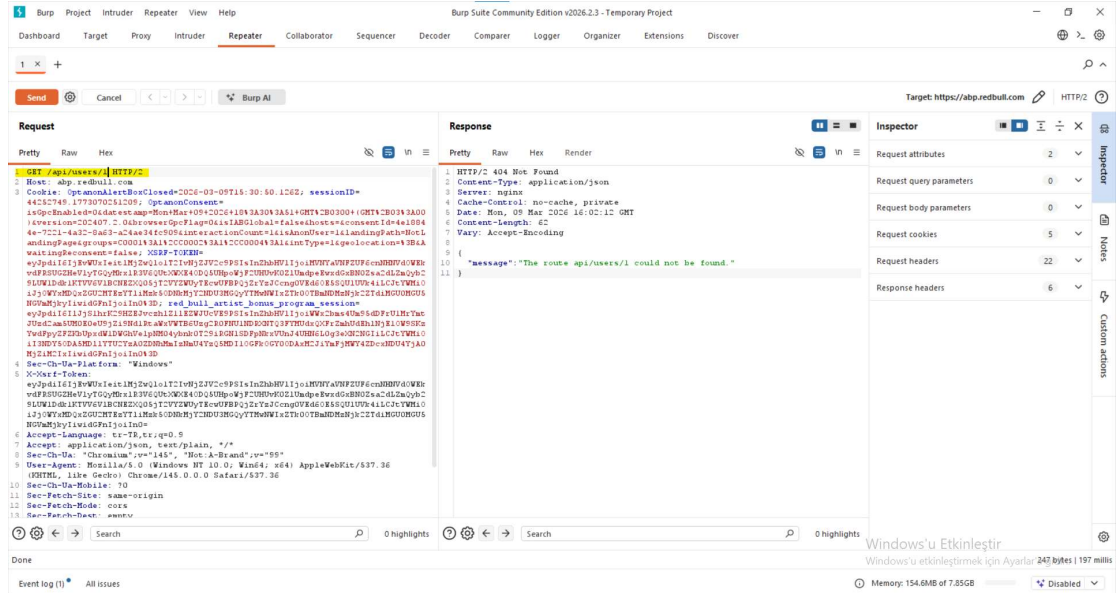
Cephe 2: Zafiyet Avı (Vulnerability Assessment)

Recon (keşif) aşamasında bulduğum abp.redbull.com sitesini daha yakından incelemek için Burp Suite aracını kullandım. Buradaki amacım, sitenin dışarıdan gelen hatalı veya kötü niyetli isteklere karşı ne kadar hazırlıklı olduğunu test etmektir. İlk olarak, giriş formları üzerinden yapılan ve siber güvenlikte çok yaygın olan XSS (Cross-Site Scripting) denemesini gerçekleştirdim. Siteye bir kullanıcı adı yerine <script>alert(1)</script> gibi bir kod göndererek sistemin kafasını karıştırmaya çalıştım. Ancak sistem bu durumu hemen fark etti ve güvenli bir şekilde girişimi engelleyerek bana "Erişim Reddedildi" mesajını verdi.

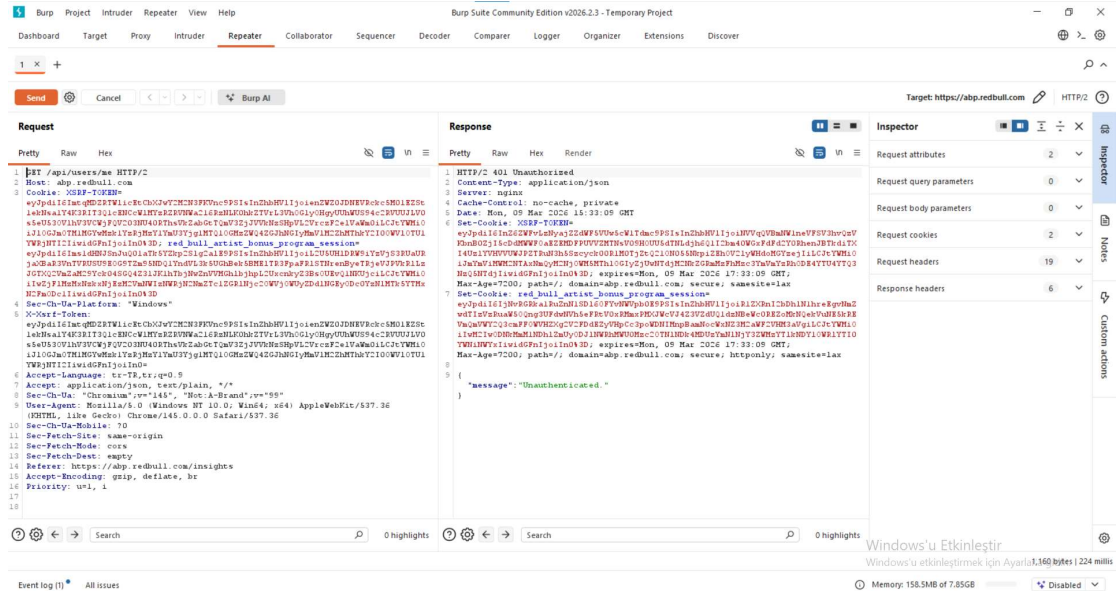


Daha sonra, sitenin adres satırındaki veya arka plandaki parametrelerle oynayarak başkasına ait verilere ulaşım test ettim. Buna güvenlik literatüründe IDOR deniyor.

Ben /api/users/me (yani "benim bilgilerimi getir") diyen adresteki me kısmını silip yerine rastgele 1 rakamını yazdım. Amacım, sistemin bana 1 numaralı kullanıcının bilgilerini verip vermeyeceğini görmektir. Fakat sistem bu isteği de geri çevirdi ve "Böyle bir sayfa/rota bulunamadı" (404 Not Found) diyerek güvenlik duvarını aşmama izin vermedi.



Son olarak, uygulamanın oturum güvenliğini nasıl sağladığına baktım. Sistemin her isteğe özel "Token" ve "Cookie" (çerez) dediğimiz dijital anahtarlar eklediğini gördüm. Bu anahtarlar olmadan veya yanlış bir anahtarla içeri girmeye çalıştığımda, sistem beni "Yetkisiz Kullanıcı" (401 Unauthorized) olarak işaretleyip dışarıda bıraktı. Yaptığım tüm bu testler, sitenin güvenlik önlemlerinin düzgün çalıştığını ve temel saldırı yöntemlerine karşı korunaklı olduğunu bana kanıtlamış oldu.



BÖLÜM D

Zafiyetlerin Etki Analizi ve CVSS Mantığı: İlk senaryoda karşılaştığımız iki farklı XSS zafiyetini kıyasladığımda, bunların şirkete vereceği zararın birbirinden tamamen farklı olduğunu görüyorum. "Hakkımızda" sayfasındaki Reflected XSS, sadece saldırganın hazırladığı özel bir linke tıklayan kullanıcıyı etkilediği için etkisi sınırlıdır. Ancak "Admin Paneli"ndeki Stored XSS çok daha tehlikelidir; çünkü bu zararlı kod sisteme bir kez kaydedilir ve o sayfayı açan her yöneticinin bilgilerini otomatik olarak çalar. Bir VDP yetkilisine bu durumu açıklarken, Admin panelindeki açığın tüm sistemi ele geçirme riski taşıdığını, bu yüzden de CVSS skorunun "Kritik" seviyede olması gerektiğini, diğerinin ise sadece bireysel bir risk olduğunu belirtirim.

SQL Injection ve Güvenli Yazılım Geliştirme: İkinci senaryoda yazılım ekibinin sadece "tek tırnak" (') gibi karakterleri yasaklayarak (Blacklisting) aldığı önlemi incelediğimde, bunun kalıcı bir çözüm olmadığını saptadım. Bir mühendis olarak biliyorum ki, saldırganlar farklı kodlama yöntemleriyle bu yasağı kolayca geçebilirler; bu durum sadece ön kapıyı kilitleyip pencereleri açık bırakmak gibidir. Bu sorunu kökten çözmek için yazılımcılara "Prepared Statements" (Parametrik Sorgular) mimarisini kullanmalarını öneririm. Bu yöntemin temel mantığı, kullanıcıdan gelen veriyi sistem komutlarından tamamen ayırmaktır. Bu sayede veritabanı, gelen veriyi ne olursa olsun bir "emir" değil, sadece basit bir "metin" olarak görür ve saldırı girişimlerini en baştan etkisiz hale getirir.